

Batch 3 — Schedulers, Context and Execution Control

Goal: understand where reactive code actually runs, how threads change across a request, how request-scoped data survives those thread hops, and how to delay source creation correctly.

0. Execution Mental Model First

Batch 1 taught you **when execution starts**.

Batch 2 taught you **what operators do after execution starts**.

Batch 3 teaches you **where execution runs** and **how request metadata survives that journey**.

This is where many senior interviews become tricky, because candidates know operators but still cannot answer:

- Which thread is running this part?
- Why did execution move off the Netty event loop?
- Why did `ThreadLocal` stop working?
- Why did `Mono.just(...)` run too early?
- Why did `subscribeOn(...)` not behave the way I expected?

The Four Questions To Ask For Any Reactive Request

1. Which thread subscribed to the source?
2. Which thread is delivering the current signal?
3. Did I intentionally shift execution with a scheduler?
4. If data like `traceId` or auth token must survive thread hops, where is it stored?

The Core Rules Of Batch 3

- By default, operators run on the thread that delivers the signal.
- `Scheduler` is not "business logic." It is execution control.
- `publishOn(...)` moves downstream execution from that point onward.
- `subscribeOn(...)` influences where subscription and upstream source work start.
- `ThreadLocal` is unreliable in reactive flows because the same request can touch multiple threads.
- `Context` is the reactive replacement for thread-bound request metadata.
- `defer(...)` and `fromCallable(...)` help you keep source creation lazy and correctly timed.

Simple Analogy

Think of a courier network:

- operators are processing stations
- schedulers decide which lane or team handles the next stations
- context is the tracking label attached to the package itself
- `defer(...)` means do not even prepare the package until the shipment actually starts

Decision Cheat Sheet

- CPU-bound synchronous work: `Schedulers.parallel()`
 - Blocking bridge to legacy client or SDK: `Schedulers.boundedElastic()`
 - One-thread serialization: `Schedulers.single()`
 - No thread switch / run on current thread: `Schedulers.immediate()`
 - Move downstream execution: `publishOn(...)`
 - Move subscription and upstream source start: `subscribeOn(...)`
 - Carry trace/auth data across thread hops: `Context`
 - Create publisher lazily per subscriber: `defer(...)`
 - Wrap blocking call lazily into `Mono`: `fromCallable(...)`
-

11. Schedulers

What They Change In The Request Flow

Schedulers decide **where** a portion of the reactive pipeline runs.

Without a scheduler switch:

- Netty event loop or current signal thread continues driving the pipeline

With a scheduler switch:

- work moves to another execution pool suitable for the kind of task being done

This matters because WebFlux is safe only if you do not accidentally run blocking or heavy CPU work on the wrong thread.

Simple Analogy

Think of a hospital triage system:

- event loop threads are the emergency desk handling fast routing and coordination
- `parallel()` is the specialist team for CPU-heavy analysis
- `boundedElastic()` is the back-office team that can spend time waiting on slow paperwork
- `single()` is one dedicated clerk doing one thing in order

You do not send every kind of work to the emergency desk.

Common Scheduler Types

`Schedulers.immediate()`

- uses the current thread
- no real scheduling hop
- mostly useful for controlled cases or tests

`Schedulers.single()`

- one reusable thread
- useful when you need serialized execution

- not for blocking high-latency workloads

Schedulers.parallel()

- fixed-size pool aimed at CPU-bound work
- good for short, compute-heavy transformations
- bad for blocking I/O

Schedulers.boundedElastic()

- designed for blocking work that you cannot avoid
- good for JDBC bridge, old SDK, filesystem calls, legacy HTTP client, crypto library that blocks
- much safer than blocking the Netty event loop

Why Scheduler Choice Matters

Suppose request needs:

- two non-blocking WebClient calls
- one legacy blocking credit-check SDK call
- a CPU-heavy response enrichment step

Correct strategy is usually:

- leave non-blocking HTTP on Netty/reactive threads
- isolate blocking SDK on `boundedElastic()`
- move heavy CPU transformation to `parallel()` only if it is genuinely expensive

What Senior Interviewers Want To Hear

Schedulers are not performance magic.

They are isolation tools.

If you put blocking work on `parallel()`, you can starve CPU workers.

If you keep blocking work on the event loop, you can stall many requests.

Backfires That Might Occur

- using `parallel()` for blocking database or HTTP calls
- overusing scheduler hops for trivial work and creating unnecessary complexity
- thinking `boundedElastic()` makes blocking code fully reactive
- hiding architectural problems by scheduling everything away instead of fixing root causes

Anti-Patterns

- `.publishOn(Schedulers.parallel())` around blocking I/O
- wrapping every chain with scheduler changes without knowing why
- moving simple DTO mapping off the event loop when it is tiny and cheap
- assuming more scheduler hops always means more scalability

Code Sample (Java)

```
Mono<CustomerRiskView> riskView = customerClient.getCustomer(customerId)
    .zipWith(Mono.fromCallable(() -> legacyRiskSdk.fetchRisk(customerId))
        .subscribeOn(Schedulers.boundedElastic()))
    .publishOn(Schedulers.parallel())
    .map(tuple -> new CustomerRiskView(tuple.getT1(), tuple.getT2()));
```

What happens here:

- `customerClient.getCustomer(...)` stays in non-blocking reactive flow
- `legacyRiskSdk.fetchRisk(...)` runs on `boundedElastic()` because it blocks
- final mapping moves to `parallel()` if the enrichment is heavy enough to justify it

Interview Trap

"`parallel()` is the right scheduler for any slow task because it gives more threads."

That is wrong. `parallel()` is for CPU-bound work, not blocking waits.

Quick Revision Notes

- scheduler = where work runs, not what work means
- `boundedElastic()` = blocking bridge
- `parallel()` = CPU-bound work
- `single()` = serialized one-thread flow
- Trap to remember = scheduler choice is about isolation and fit, not blind speed

12. `publishOn` vs `subscribeOn`

What They Change In The Request Flow

This is one of the most frequently misunderstood parts of Reactor.

They both affect threading, but they affect different parts of the flow.

- `subscribeOn(...)` affects where subscription and upstream source work start
- `publishOn(...)` affects where downstream operators continue from that point onward

Simple Analogy

Imagine a factory belt:

- `subscribeOn(...)` decides which team starts the conveyor belt at the source end
- `publishOn(...)` says, "From this station onward, move the package to another team"

The Easiest Mental Model

`subscribeOn(...)`

Think upstream.

- source creation
- subscription signal
- request to source

It is about where the pipeline begins execution.

publishOn(...)

Think downstream.

- operators after this point
- their `map`, `flatMap`, `doOnNext`, etc.

It is about where later signals are processed.

Example 1: subscribeOn(...)

```
Mono<String> pipeline = Mono.fromCallable(() -> {
    log.info("source thread = {}", Thread.currentThread().getName());
    return legacyClient.fetch();
})
    .subscribeOn(Schedulers.boundedElastic())
    .map(value -> value.toUpperCase());
```

What this means:

- the callable source starts on `boundedElastic()`
- upstream source execution is moved away from the event loop
- downstream may still continue on that same scheduler unless another operator shifts it again

Example 2: publishOn(...)

```
Mono<String> pipeline = webClient.get()
    .uri("/profile")
    .retrieve()
    .bodyToMono(String.class)
    .publishOn(Schedulers.parallel())
    .map(body -> body + "-processed");
```

What this means:

- WebClient response arrives in reactive networking flow
- after `publishOn(...)`, later operators like `map(...)` run on `parallel()`

Example 3: Both Together

```
Mono<Summary> pipeline = Mono.fromCallable(() -> legacyClient.fetch(customerId))
    .subscribeOn(Schedulers.boundedElastic())
    .publishOn(Schedulers.parallel())
    .map(this::heavyTransform);
```

Meaning:

- blocking source runs on `boundedElastic()`
- heavy CPU transform runs on `parallel()`

Important Rule About Multiple `publishOn(...)`

Multiple `publishOn(...)` operators are allowed.

Each one changes the thread for the downstream section after it.

Important Rule About `subscribeOn(...)`

In normal interview language, only one effective `subscribeOn(...)` matters for a given source chain.

The practical point is:

- adding multiple `subscribeOn(...)` operators usually does not do what candidates expect
- use one clear `subscribeOn(...)` near the source you want to control

Why This Matters In Request Flow

Suppose request flow is:

1. receive HTTP request on event loop
2. call non-blocking customer service
3. call blocking legacy fraud SDK
4. run expensive ranking algorithm
5. serialize response

Then a good shape is:

- keep customer service reactive
- put fraud SDK behind `fromCallable(...).subscribeOn(boundedElastic())`
- use `publishOn(parallel())` before expensive ranking if truly needed

Backfires That Might Occur

- using `publishOn(...)` and thinking it changes the source thread retroactively
- using `subscribeOn(...)` late in the chain and not understanding why it still affects the source
- stacking many thread hops and making traces harder to reason about
- using scheduler switches to paper over bad blocking architecture

Anti-Patterns

- `publishOn(...)` expecting upstream WebClient call itself to move
- several `subscribeOn(...)` operators added randomly
- thread hopping every two lines with no measurable benefit
- forgetting that thread hops make `ThreadLocal` unreliable

Code Sample (Java)

```
Mono<CustomerView> customerView = customerClient.getCustomer(customerId)
    .publishOn(Schedulers.parallel())
    .map(this::expensiveScoreComputation)
    .zipWith(Mono.fromCallable(() -> legacyPreferencesSdk.fetch(customerId))
        .subscribeOn(Schedulers.boundedElastic()))
    .map(tuple -> new CustomerView(tuple.getT1(), tuple.getT2()));
```

Interview Trap

"`publishOn(...)` and `subscribeOn(...)` are interchangeable thread-switch operators."

That is wrong. `publishOn(...)` affects downstream processing. `subscribeOn(...)` affects subscription and upstream source start.

Quick Revision Notes

- `subscribeOn` = move source start / upstream subscription work
- `publishOn` = move downstream operators from that point forward
- multiple `publishOn` hops are allowed
- keep `subscribeOn` use simple and intentional
- Trap to remember = `publishOn` does not retroactively move upstream work

13. Reactor Context

What It Changes In The Request Flow

Reactive requests often move across threads.

That breaks the old assumption behind `ThreadLocal` :

- one request -> one thread -> request data lives in that thread

In WebFlux, one request may touch:

- Netty event loop thread
- `boundedElastic()` thread for blocking bridge
- `parallel()` thread for CPU work
- another event loop thread delivering later signals

So request metadata like:

- `traceId`
- correlation id
- auth token
- tenant id
- locale

cannot safely depend on staying in one Java thread.

Reactor `Context` solves this by attaching metadata to the reactive subscriber chain instead of to a thread.

Simple Analogy

ThreadLocal is like writing a note on a worker's hand.

Reactor Context is like attaching a shipping label to the package itself.

If the package moves to another worker, the label stays with the package.

Core Idea

`Context` is an immutable key-value map propagated with the reactive chain.

You commonly:

- write into it with `contextWrite(...)`
- read from it with `deferContextual(...)`

Why ThreadLocal Fails Here

Suppose you set `ThreadLocal<String> traceId = "abc"` on one event loop thread.

If later a signal is processed on `boundedElastic()` or `parallel()`, that other thread does not automatically carry the old ThreadLocal value.

Now logging and tracing become inconsistent.

Basic Usage Pattern

```
Mono<String> pipeline = Mono.deferContextual(ctx -> {
    String traceId = ctx.getOrDefault("traceId", "missing");
    return Mono.just("traceId=" + traceId);
})
    .contextWrite(context -> context.put("traceId", "req-123"));
```

Why This Matters In Request Flow

Imagine a request filter adds a trace id.

Later the request:

- calls WebClient
- switches to `boundedElastic()` for a blocking legacy step
- switches again to `parallel()` for expensive enrichment

If you rely on ThreadLocal, trace data may disappear after thread hops.

If you rely on Reactor Context, the trace id still travels with the reactive chain.

What Context Is Good For

- tracing and correlation IDs
- auth or tenant metadata used by downstream filters
- MDC integration through supported logging/tracing infrastructure
- request-scoped flags that should travel with the chain

What Context Is Not Good For

- large business objects
- replacing normal data flow in method parameters
- mutable shared state
- storing entire domain aggregates as hidden side data

Backfires That Might Occur

- trying to read request data from ThreadLocal after a scheduler hop
- overusing Context for business payload instead of explicit method arguments
- assuming Context is mutable shared state
- forgetting to write context before the section that needs it

Anti-Patterns

- ThreadLocal -only correlation in reactive APIs
- hiding important business inputs only inside Context
- storing big objects in Context and making debugging harder
- mutating request state outside explicit reactive flow assumptions

Code Sample (Java)

```
Mono<CustomerAuditView> view = Mono.deferContextual(ctx -> {  
    String traceId = ctx.getOrDefault("traceId", "missing");  
  
    return customerClient.getCustomer(customerId)  
        .map(customer -> new CustomerAuditView(customer, traceId));  
})  
    .contextWrite(context -> context.put("traceId", "req-42"));
```

Interview Trap

"ThreadLocal works fine in WebFlux as long as I set it at the start of the request."

That is wrong. Thread hops are normal in reactive flows, so thread-bound data is not a reliable request store.

Quick Revision Notes

- Context = subscriber-bound metadata, not thread-bound metadata
 - contextWrite(...) stores it
 - deferContextual(...) reads it lazily during execution
 - use it for tracing/auth metadata, not hidden business payload
 - Trap to remember = request may move threads, but Context travels with the chain
-

14. `defer(...)` and `fromCallable(...)`

What They Change In The Request Flow

These operators control **when source creation actually happens**.

This matters because one of the most expensive mistakes in WebFlux is triggering work too early during assembly.

Simple Analogy

- `Mono.just(value)` = package is already prepared now
- `defer(...)` = prepare the package only when shipment starts
- `fromCallable(...)` = call the supplier lazily when shipment starts, and turn success or failure into reactive signals

`defer(...)`

`defer(...)` creates the publisher lazily per subscriber.

That means the function you pass is called only when subscription happens.

```
Mono<String> timestamp = Mono.defer(() -> Mono.just("ts=" + System.currentTimeMillis()));
```

Each subscription gets a fresh publisher and a fresh timestamp.

Use `defer(...)` when:

- source must be created lazily
- you need per-subscriber freshness
- the source depends on runtime state available only at execution time
- you need to avoid assembly-time side effects

`fromCallable(...)`

`fromCallable(...)` wraps a blocking or exception-throwing function into a lazy `Mono`.

```
Mono<String> valueMono = Mono.fromCallable(() -> legacyClient.fetchValue());
```

That callable runs on subscription.

If it throws, the error becomes an `onError` signal.

If it blocks, combine it with `subscribeOn(Schedulers.boundedElastic())`.

```
Mono<String> safeBlocking = Mono.fromCallable(() -> legacyClient.fetchValue())  
    .subscribeOn(Schedulers.boundedElastic());
```

Why `defer(...)` And `fromCallable(...)` Are Different

- `defer(...)` is a lazy factory for **any publisher**
- `fromCallable(...)` is a lazy bridge from **callable value production** to `Mono`

Use `defer(...)` when you want to lazily return a whole publisher chain.

Use `fromCallable(...)` when you have one callable that produces one value or throws.

Why This Matters In Request Flow

Suppose request requires:

- reading a runtime config flag
- maybe calling a legacy blocking SDK
- making sure work does not start before Spring subscribes

Then:

- `defer(...)` ensures the decision and source creation happen at execution time
- `fromCallable(...)` ensures the blocking function call happens lazily and can be isolated properly

The Classic Bad Example

```
Mono<String> wrong = Mono.just(legacyClient.fetchValue());
```

Problem:

- `fetchValue()` runs immediately during assembly
- current thread blocks immediately
- controller may be stuck before the reactive pipeline is even returned

Better Examples

```
Mono<String> right1 = Mono.defer(() -> tokenService.refreshTokenMono());

Mono<String> right2 = Mono.fromCallable(() -> legacyClient.fetchValue())
    .subscribeOn(Schedulers.boundedElastic());
```

Backfires That Might Occur

- using `Mono.just(blockingCall())` and losing laziness
- forgetting `subscribeOn(boundedElastic())` for blocking callables
- using `defer(...)` unnecessarily where a plain reactive source is already correct
- confusing per-subscription freshness with shared cached behavior

Anti-Patterns

- immediate blocking source creation during controller assembly
- using `defer(...)` everywhere even when nothing needs lazy recreation
- assuming `fromCallable(...)` alone makes blocking code non-blocking
- forgetting that each subscription may recreate work with `defer(...)`

Code Sample (Java)

```
Mono<AuthSnapshot> authMono = Mono.defer(() -> authService.currentAuthSnapshot());

Mono<LegacyScore> scoreMono = Mono.fromCallable(() -> legacyScoreSdk.fetchScore(customerId))
    .subscribeOn(Schedulers.boundedElastic());
```

Interview Trap

"`Mono.just(blockingCall())` is fine because emission still happens only on subscription."

That is wrong. Emission may be lazy, but the blocking computation already happened earlier during assembly.

Quick Revision Notes

- `defer` = lazy publisher factory per subscriber
- `fromCallable` = lazy callable-to-Mono bridge
- blocking callables still need `boundedElastic()` isolation
- Trap to remember = lazy emission is not the same as lazy computation

Batch 3 — Putting Execution Control Into One Request Story

Now connect Batch 3 concepts into one realistic WebFlux request.

Use Case

Endpoint: `GET /customers/{id}/profile-summary`

Need to build response from:

- customer profile service via WebClient
- loyalty data via reactive DB
- legacy risk SDK that blocks
- heavy scoring and ranking logic
- trace id that must survive thread hops for logs and audit

The Code

```
@GetMapping("/customers/{id}/profile-summary")
Mono<CustomerProfileSummary> summary(@PathVariable String id) {
    Mono<Customer> customerMono = customerClient.getCustomer(id);

    Mono<Loyalty> loyaltyMono = loyaltyRepository.findByCustomerId(id);

    Mono<RiskScore> riskMono = Mono.fromCallable(() -> legacyRiskSdk.fetchRisk(id))
        .subscribeOn(Schedulers.boundedElastic());

    return Mono.deferContextual(ctx -> Mono.zip(customerMono, loyaltyMono, riskMono)
        .publishOn(Schedulers.parallel())
        .map(tuple -> buildSummary(
            tuple.getT1(),
            tuple.getT2(),
            tuple.getT3(),
            ctx.getOrDefault("traceId", "missing")
        )))
        .doOnSubscribe(sub -> log.info("summary request started for {}", id))
        .doOnSuccess(result -> log.info("summary request succeeded for {}", id))
        .doOnError(ex -> log.error("summary request failed for {}", id, ex))
        .doFinally(signal -> metrics.recordSummaryCompletion(id, signal))
        .contextWrite(context -> context.put("traceId", "req-" + id));
}
```

Why Each Batch 3 Concept Is Here

- `fromCallable(...)` = wrap lazy blocking risk SDK call
- `subscribeOn(boundedElastic())` = move blocking SDK off the event loop
- `Mono.deferContextual(...)` = read request metadata from Reactor Context during execution
- `contextWrite(...)` = store trace id in the reactive chain
- `publishOn(parallel())` = move heavy summary building away from event-loop or I/O threads

Full Request Life Journey

Phase 1: Request Enters WebFlux

1. Browser calls `GET /customers/42/profile-summary`
2. Netty receives request on an event loop thread
3. Spring routes request to controller

Phase 2: Assembly Happens

Controller method builds:

- `customerMono`
- `loyaltyMono`
- `riskMono` with `fromCallable(...).subscribeOn(boundedElastic())`
- outer `deferContextual(...)`

- `zip(...)`
- `publishOn(parallel())`
- `map(...)`
- `doOn...` hooks
- `contextWrite(...)`

At this moment:

- WebFlux has the recipe
- risk SDK has not run yet
- summary object has not been built yet
- trace id has not yet been read from Context

Phase 3: Spring Subscribes

After controller returns `Mono<CustomerProfileSummary>`, Spring subscribes because it now needs actual data to write the HTTP response.

Execution begins.

Phase 4: Context Becomes Available

`contextWrite(...)` attaches `traceId` to the reactive chain.

When `deferContextual(...)` executes, it can read that `traceId` even if later thread hops happen.

This is the precise reason Context exists.

Phase 5: Reactive Sources Start

- `customerClient.getCustomer(id)` begins non-blocking outbound HTTP I/O
- `loyaltyRepository.findById(id)` begins reactive DB query
- `riskMono` subscribes and schedules `legacyRiskSdk.fetchRisk(id)` on `boundedElastic()`

This is a major moment:

- customer HTTP and reactive DB stay in non-blocking flow
- legacy blocking SDK is isolated to a worker built for blocking waits
- the Netty event loop is not parked for the legacy SDK call

Phase 6: While Waiting

Suppose timing is:

- loyalty DB = 150 ms
- customer service = 300 ms
- legacy risk SDK = 2.5 seconds

During that 2.5-second risk wait:

- no event loop thread is blocked just to wait
- a bounded-elastic worker is waiting on the blocking SDK
- Netty event loop keeps handling other requests and socket events

That is the exact reason scheduler choice matters.

Phase 7: Results Arrive On Different Threads

- loyalty result may arrive via reactive DB driver thread/signal path
- customer result may arrive via Reactor Netty I/O path
- risk result arrives from `boundedElastic()` when the blocking call finishes

Now you have one request whose signals touched multiple execution contexts.

This is why `ThreadLocal` is unsafe, but `Context` still works.

Phase 8: `zip(...)` Coordinates Results

`zip(...)` waits for:

- customer
- loyalty
- risk

Only when all three are ready does it emit downstream.

Again, the total response latency is still dominated by the slowest required dependency.

WebFlux did not make the legacy SDK faster.

It prevented the wrong threads from being blocked while that slow work was happening.

Phase 9: `publishOn(parallel())` Moves Heavy Mapping

Once `zip(...)` emits, `publishOn(parallel())` moves the downstream `map(...)` to the CPU-oriented scheduler.

Why?

Because `buildSummary(...)` is assumed to be heavy enough that we do not want it running on I/O-oriented threads.

If summary mapping were trivial, this extra scheduler hop might not be worth it.

That is the senior answer: scheduler shifts are justified by workload, not habit.

Phase 10: Context Is Read During Mapping

Inside `deferContextual(...)`, the code reads:

- `traceId`

That trace id is still available even though signals may have crossed:

- event loop thread
- `boundedElastic` worker
- parallel worker

That is the key Context guarantee.

Phase 11: Response Is Written

- final `CustomerProfileSummary` is emitted
- Spring serializes it to JSON
- HTTP response bytes are written back to the client
- `doFinally(...)` records completion metrics

What Would Go Wrong Without Batch 3 Concepts?

- without `fromCallable(...)`, the legacy SDK might run too early or outside reactive flow
- without `subscribeOn(boundedElastic())`, blocking risk lookup could stall the event loop or wrong thread
- without `Context`, trace data could disappear after thread hops
- without understanding `publishOn(...)`, heavy mapping might run on I/O threads accidentally
- without understanding `defer(...)`, source creation might happen at assembly time instead of execution time

Final Batch 3 Memory Model

Use this sentence in interviews:

"Batch 3 is about execution control: schedulers choose the right place for work, `publishOn` and `subscribeOn` control where different phases run, `Context` carries request metadata across thread hops, and `defer` / `fromCallable` ensure source creation happens at the correct time."

Batch 3 — Interview Hot Questions

1. Why is `boundedElastic()` preferred for blocking code?

Because it isolates blocking waits away from the Netty event loop and CPU-oriented pools.

2. Why is `parallel()` the wrong choice for blocking I/O?

Because it is meant for CPU-bound work, and blocking it can starve compute tasks.

3. What is the most important difference between `publishOn(...)` and `subscribeOn(...)` ?

`subscribeOn(...)` influences source/upstream start. `publishOn(...)` shifts downstream execution from that point onward.

4. Why does `ThreadLocal` break in `WebFlux`?

Because one request can cross multiple threads, so thread-bound request data is no longer reliable.

5. What does `Reactor Context` solve?

It carries request-scoped metadata with the subscriber chain instead of relying on a single thread.

6. When should I use `defer(...)` ?

When a publisher must be created lazily per subscriber, especially if it depends on runtime state or must avoid assembly-time execution.

7. When should I use `fromCallable(...)` ?

When you have a lazy one-value source from blocking or exception-throwing code and want it represented as a `Mono`.

8. Does `fromCallable(...)` alone make blocking code non-blocking?

No. It makes it lazy and reactive, but blocking work still needs a proper scheduler like `boundedElastic()`.

Batch 3 — Revision Notes

- One-line summary: Batch 3 is about controlling where reactive work runs and how request metadata survives when execution moves across threads.
- Three keywords: scheduler, thread hop, context.
- One trap: confusing lazy source creation with non-blocking execution.
- One memory trick: scheduler chooses the lane, `publishOn` moves the next stations, `subscribeOn` starts the source lane, Context travels with the package, `defer` waits to pack the box.